

Git + VS Code + GitHub Troubleshooting Guide

Consolidated troubleshooting guide based on publishing YAML projects

Git + GitHub: Issues We Encountered and Commands Used

1. Final architecture we achieved

Initially, every YAML project had its own .git repository.

✗ Initial situation

```
YAML-Learning
├── .git
├── Load-All-YAML-Files
│   └── .git
├── Stage 3...
│   └── .git
├── Stage 3.1...
│   └── .git
├── YAML-Exploration
│   └── .git
├── YAML-Practicals
│   └── .git
└── YAML-Stage3.2...
    └── .git
```

This means you actually had 7 Git repositories.

✓ Final situation

```
YAML-Learning
├── .git ← ONE repository
├── Load-All-YAML-Files
├── Stage 3 YAML-Based Application Configuration
├── Stage 3.1 YAML-Based Application Configuration
├── YAML-Exploration
├── YAML-Practicals
└── YAML-Stage3.2-Pydantic-Settings
```

And this entire folder is connected to:

```
GitHub
└── yaml-learning-projects
```

This is the correct structure for your learning repository.

2. Issue: Confusion between PowerShell and Git commands

We initially tried to find .git folders.
The command was:

```
Get-ChildItem -Force -Directory -Recurse -Filter ".git"
```

What is Get-ChildItem?

It is not a Git command.

It is a PowerShell command used to list files and directories.

`Get-ChildItem` is similar to: `dir` or: `ls` in other environments.

`-Force` : Shows hidden files/folders as well. This was important because .git is normally hidden.

`-Directory` : Only searches directories.

`-Recurse` : Searches inside subdirectories.

`-Filter ".git"` : Looks specifically for directories named .git.

3. Issue: Accidentally typing the PowerShell prompt

You encountered an error similar to:

```
D:\AI : The term 'D:\AI' is not recognized...
```

The problem was entering something like:

```
D:\AI Learning\YAML-Learning>Get-ChildItem
```

The prompt itself: `PS D:\AI Learning\YAML-Learning>` is not part of the command.

You should type only: `Get-ChildItem`

Rule to remember:

If PowerShell shows: `PS D:\AI Learning\YAML-Learning>`

you type: `git status`

not: `PS D:\AI Learning\YAML-Learning>git status`

4. Issue: Accidental files were created

Because of the earlier command-entry mistakes, Git showed:

```
'Get-ChildItem'  
Get-ChildItem  
operable
```

These were not YAML projects.

We inspected the directory with:

```
Get-ChildItem -Force
```

and discovered:

```
-a--- 'Get-ChildItem'  
-a--- Get-ChildItem  
-a--- operable
```

They were accidental files.

We removed them with:

```
Remove-Item -Force ".\Get-ChildItem"  
Remove-Item -Force ".\Get-ChildItem"  
Remove-Item -Force ".\operable"
```

`Remove-Item` : PowerShell command used to delete files/folders.

`-Force` : Forces deletion when necessary.

5. Issue: Multiple .git repositories

We discovered the major structural problem with:

```
dir -Force -Recurse -Directory -Filter .git
```

Initially it showed:

```
YAML-Learning\.git  
YAML-Learning\Load-All-YAML-Files\.git  
YAML-Learning\Stage 3...\git  
YAML-Learning\Stage 3.1...\git  
YAML-Learning\YAML-Exploration\.git  
YAML-Learning\YAML-Practicals\.git  
YAML-Learning\YAML-Stage3.2...\git
```

Problem: Each project was an independent Git repository.

Solution: We kept `YAML-Learning\.git` and removed the nested .git folders.

For example:

```
Remove-Item -Recurse -Force ".\Load-All-YAML-Files\.git"
```

The same approach was used for the other projects.

Important lesson:

For your setup: One parent learning repository → many project folders.

Do not initialize Git separately inside every project.

6. Issue: Understanding git status

We repeatedly used: `git status`

This is one of the most important Git commands.

It tells you: Current branch, Modified files, Untracked files, Staged files, Whether there are pending changes.

Initially we saw:

```
On branch master
No commits yet

Untracked files:
  Load-All-YAML-Files/
  ...
```

This meant: Git knows these folders exist, but they haven't been added to the repository yet.

Later we reached:

```
On branch main
nothing to commit, working tree clean
```

This means: Everything currently in the repository has been committed.

7. Issue: Creating the .gitignore

We created a root .gitignore.

Its purpose is to prevent unnecessary or sensitive files from being uploaded.

For your Python projects, important entries were:

```
__pycache__/  
*.py[cod]  
  
.venv/  
venv/  
env/  
  
.env  
.env.*  
  
.vscode/  
  
.DS_Store  
Thumbs.db
```

This is especially important because your YAML learning project is moving toward .env and secrets.

8. git add .

Command: `git add .`

Purpose: Moves changes from Working Directory → Staging Area.

It tells Git: "Prepare all current changes for my next commit."

We then checked `git status` to verify exactly what would be committed.

Important habit: Don't blindly use `git add .` without checking `git status`, especially when learning.

9. git commit

We created the commit:

```
git commit -m "Add YAML learning projects"
```

Result:

```
810268f Add YAML learning projects
```

Purpose: A commit creates a permanent snapshot in your local Git history.

Think: Files → git add → Staging → git commit → Local Git history.

A commit is not yet on GitHub.

10. git branch

We used `git branch` to check the current branch.

Initially the repository was on: `master`

We changed it to: `main` using:

```
git branch -M main
```

`-M` : Renames the current branch to main, forcing the rename if necessary.

11. git remote -v

We used: `git remote -v`

Purpose: Shows which remote GitHub repository your local repository is connected to.

Initially, `git remote -v` returned nothing.

That meant: Local Git repository exists, but it isn't connected to GitHub.

12. git remote add origin

We initially configured the wrong URL:

```
https://github.com/techgurupv/YAML-Learning.git
```

The actual repository was:

```
https://github.com/techgurupv/yaml-learning-projects.git
```

The origin name is simply the conventional name for your primary remote repository.

Typical configuration: origin → GitHub repository

13. Issue: Wrong GitHub repository URL

We tried `git push -u origin main` and received:

```
remote: Repository not found.  
fatal: repository ... not found
```

Root cause: The configured URL was `YAML-Learning.git` but your actual repository was `yaml-learning-projects.git`.

Solution: We changed the remote:

```
git remote set-url origin https://github.com/techgurupv/yaml-learning-projects.git
```

Then `git remote -v` confirmed the correct URL.

14. git ls-remote

We used: `git ls-remote origin`

Purpose: Tests whether your local Git can communicate with the remote repository and retrieves the remote references.

We got:

```
5006c39... HEAD
5006c39... refs/heads/main
```

That confirmed: Local Git → GitHub  connection working.

This is a very useful diagnostic command when git push isn't working.

15. Issue: Existing GitHub repository had history

Before deleting the old repository, we discovered that:

Local: 810268f Add YAML learning projects

while GitHub had: 5006c39 Existing project

These were different Git histories. Trying to push directly could have produced a `rejected / fetch first` error.

The normal solution would be `git fetch origin` followed by `git merge origin/main --allow-unrelated-histories`.

But in your case, you decided to delete the old GitHub repository and create a fresh empty repository, which was simpler because you wanted a clean learning repository.

16. git fetch

We discussed: `git fetch origin`

Purpose: Downloads information/commits from GitHub into your local Git repository without modifying your working files.

Think: GitHub → git fetch → Local Git knowledge of remote history.

It is different from git pull.

17. Why we didn't use git push --force

We explicitly avoided `git push --force` because the existing GitHub repository contained history. `--force` can overwrite remote history.

Rule: Avoid force push unless you fully understand the consequences.

For your learning repository, normal `git push` is what you want.

18. Final clean repository

After deleting the old GitHub repository, you created a fresh repository.

Your local repository already had: `810268f Add YAML learning projects`

So you didn't need to create another commit.

You simply connected the new remote and pushed:

```
git push -u origin main
```

`-u` : Sets the upstream tracking relationship: local main ↕ origin/main.

After that, future pushes can usually be simply: `git push` instead of `git push -u origin main`.

19. Complete command reference

Here is your Git command cheat sheet.

Command	Purpose
<code>git init</code>	Creates a new Git repository
<code>git status</code>	Shows current repository state
<code>git add .</code>	Stages all changes
<code>git add <file></code>	Stages a specific file
<code>git commit -m "message"</code>	Creates a local commit
<code>git log --oneline</code>	Shows commit history
<code>git branch</code>	Shows branches
<code>git branch -M main</code>	Renames current branch to main
<code>git remote -v</code>	Shows configured remotes
<code>git remote add origin URL</code>	Adds a remote
<code>git remote set-url origin URL</code>	Changes remote URL
<code>git ls-remote origin</code>	Tests/reads remote references
<code>git fetch origin</code>	Downloads remote history without merging
<code>git merge origin/main</code>	Merges remote branch into local branch
<code>git push</code>	Uploads commits to remote
<code>git push -u origin main</code>	First push + sets upstream
<code>git pull</code>	Fetch + merge remote changes
<code>git clone URL</code>	Downloads an existing repository
<code>git diff</code>	Shows unstaged changes
<code>git diff --staged</code>	Shows staged changes

20. PowerShell commands we used

These are not Git commands:

Command	Purpose
dir	Lists files/folders
Get-ChildItem	Lists files/folders
Get-ChildItem -Force	Includes hidden files
Get-ChildItem -Recurse	Searches subdirectories
Get-Item	Gets information about a file/folder
Remove-Item	Deletes files/folders
cls	Clears terminal screen

21. Your recommended workflow from now on

This is the most important part.
Your repository is now:

```
D:\AI Learning\YAML-Learning
|
├─ .git
├─ Project-1
├─ Project-2
├─ Project-3
└─ ...
```

When you create a new YAML project, simply create it under YAML-Learning.

Don't do this:

```
cd "New-Project"
git init
```

Instead:

```
YAML-Learning
└─ New-Project
```

Then from the parent repository:

```
cd "D:\AI Learning\YAML-Learning"
```

Check: `git status`

Stage: `git add .`

Commit: `git commit -m "Add new YAML project"`

Push: `git push`

That's your normal workflow.

The Git mental model to remember

Think of Git as having three levels:



The core workflow to memorize is: **edit** → **status** → **add** → **commit** → **push**

```
git status
git add .
git commit -m "Describe the change"
git push
```

That is all you need for your day-to-day YAML learning repository.